

DEV2 – JAVL – Laboratoire Java

TD 15 – Interactions et try-catch

Résumé

Au terme de ce TD vous serez capable de :

- ▷ intercepter (attraper) une exception
- ▷ lire un fichier ou l'entrée standard de manière robuste
- ▷ distinguer les exceptions contrôlées (checked exceptions) et non-contrôlées (unchecked exceptions)

Table des matières

1 Informations théoriques	1
2 Lectures robustes au clavier	2
2.1 InputMismatchException	2
2.2 catch	2
2.3 Une technique robuste de lecture d'entier	2
2.4 Une méthode robuste de lecture d'entier	2
2.5 Lecture robuste d'un entier entre deux valeurs	3
2.6 Lecture robuste d'une chaîne parmi des choix prédéfinis	4
3 Lecture d'un fichier	4
3.1 Plusieurs clauses catch	4
3.2 La mère de toutes les exceptions	5
4 Mise en œuvre	5

1 Informations théoriques

Un `try-catch` est une construction permettant d'intercepter une exception. La syntaxe est :

```
try {  
    // du code qui pourrait lever une exception  
} catch (ClasseOuInterfaceDeLException identifiant) {  
    // du code pour gérer l'exception  
}
```

Un code est *robuste* s'il ne plante pas malgré d'éventuelles entrées erronées de la personne utilisant le système.

Par exemple une *méthode robuste de lecture d'entier* est une méthode qui retourne un entier entré au clavier, sans faire planter le programme (notamment si c'est autre chose qu'un entier qui a été entré).

L'objectif est de mettre ces concepts en œuvre.

2 Lectures robustes au clavier

2.1 InputMismatchException

```
final Scanner IN = new Scanner(System.in);
System.out.print("Veuillez entrer un entier : ");
int n = IN.nextInt();
System.out.println("Vous avez entré l'entier " + n);
System.out.println("Programme terminé");
```

Si ce n'est pas un entier qui est entré (essayez !), une `InputMismatchException` est levée par le code, et le programme se termine abruptement (il plante *sans* afficher `Vous avez entré l'entier ... ni Programme terminé`).

2.2 catch

Pour éviter le *plantage*, nous pouvons enrober l'appel dans un try-catch :

```
final Scanner IN = new Scanner(System.in);
System.out.print("Veuillez entrer un entier : ");
try {
    int n = IN.nextInt();
    System.out.println("Vous avez entré l'entier " + n);
} catch (InputMismatchException ex) {
    System.out.println("Vous n'avez pas entré d'entier");
}
System.out.println("Programme terminé");
```

Bien que techniquement robuste (le programme ne *plante* pas), nous n'avons pas pour autant une *méthode robuste de lecture d'entier* puisque le programme n'obtient finalement pas d'entier.

2.3 Une technique robuste de lecture d'entier

Une façon de s'en sortir est d'enrober le tout dans une boucle :

```
final Scanner IN = new Scanner(System.in);
boolean entierDonné = false;

while (!entierDonné) {
    System.out.print("Veuillez entrer un entier : ");
    try {
        int n = IN.nextInt();
        System.out.println("Vous avez entré l'entier " + n);
        entierDonné = true;
    } catch (InputMismatchException ex) {
        IN.nextLine(); // jette la ligne entrée à la poubelle : ce n'était pas un entier
        System.out.println("Vous n'avez pas entré d'entier");
    }
}
System.out.println("Programme terminé");
```

Essayez-le !

2.4 Une méthode robuste de lecture d'entier

À vous de jouer : adaptez le code précédent pour obtenir une *méthode* qui *retourne* l'entier lu, de façon robuste.

Lorsque c'est fait, tournez la page pour voir notre solution commentée.

```

public class Td17 {
    final static Scanner IN = new Scanner(System.in);

    public static void main(String[] args) {
        System.out.println("Vous avez entré l'entier " + readInteger());
        // ci-dessus il y a un appel inline à la méthode.
        // on aurait aussi pu introduire une variable:
        //int n = readInteger();
        // et utiliser cela.
        System.out.println("Programme terminé");
    }

    public static int readInteger() {
        while (true) { // le booléen entierDonné n'a plus d'utilité à cause du "return"
            System.out.print("Veuillez entrer un entier : ");
            try {
                return IN.nextInt();
            } catch (InputMismatchException ex) {
                IN.nextLine(); // jette la ligne entrée à la poubelle : ce n'était pas un entier
                System.out.println("Vous n'avez pas entré d'entier");
            }
        }
    }
}

```

Alternativement vous pourriez vous passer d'un try-catch :

```

public class Td17 {
    final static Scanner IN = new Scanner(System.in);

    public static void main(String[] args) {
        System.out.println("Vous avez entré l'entier " + readInteger());
        System.out.println("Programme terminé");
    }

    public static int readInteger() {
        System.out.print("Veuillez entrer un entier : ");
        while (!IN.hasNextInt()) {
            IN.nextLine();
            System.out.println("Vous n'avez pas entré d'entier");
        }
        return IN.nextInt();
    }
}

```

Comment choisir entre utiliser un try-catch et ne pas l'utiliser ? En règle générale, s'il est possible de s'en passer, il vaut mieux s'en passer. Dans le cas présent, le `Scanner` fournit la méthode `hasNextInt()` ce qui permet d'assurer que `nextInt()` ne lèvera pas d'exception : c'est donc mieux.

2.5 Lecture robuste d'un entier entre deux valeurs

À titre d'exercice, écrivez maintenant une méthode robuste de lecture d'un entier entre deux valeurs fixées.

La signature sera `int readInteger(int min, int max)` et la méthode doit re-demander jusqu'à obtenir un entier effectivement entre min et max (bornes incluses).

Attention ! Il ne faut pas *adapter* le code précédent, mais *appeler* la méthode `readInteger` qui a déjà été écrite.

2.6 Lecture robuste d'une chaîne parmi des choix prédéfinis

À titre d'exercice, écrivez maintenant une méthode robuste de lecture d'une chaîne de caractères, parmi une liste de choix prédéfinis.

La signature sera `String readString(List<String> choices)`

Aide : cette fois vous allez adapter `readInteger` (ça n'aurait pas de sens de l'appeler, n'est-ce pas ? Nous voulons des chaînes!).

Entre autres ajustements, vous devrez trouver autre chose que `hasNextInt()` comme condition, et utiliser `next()` au lieu de `nextInt()`.

3 Lecture d'un fichier

Pour lire un fichier, on peut utiliser ceci, par exemple pour lire la première ligne :

```
Files.lines(Path.of("monFichier.txt")).findFirst().get();
```

Tel quel ce code ne compile pas parce que la méthode `lines()` annonce lever une `IOException`, qui est une exception contrôlée (*checked exception*). Celle-ci doit donc obligatoirement être gérée.

Utilisons un try-catch :

```
try {
    String firstLine = Files.lines(Path.of("monFichier.txt")).findFirst().get();
    System.out.printf("Première ligne du fichier : %s%n", firstLine);
} catch (IOException e) {
    System.err.printf("Oups, une erreur : %s%n", e.toString());
}
```

En exécutant ce code, normalement vous obtenez :

```
Oups, une erreur : java.nio.file.NoSuchFileException: monFichier.txt
```

Vous constatez que le type de l'exception est `NoSuchFileException` et pas `IOException`, et pourtant l'exception a correctement été interceptée. La raison est que la première hérite de la seconde. `IOException` est donc plus générique.

3.1 Plusieurs clauses catch

On peut gérer plusieurs exceptions différentes pour gérer des situations différemment :

```
try {
    String firstLine = Files.lines(Path.of("monFichier.txt")).findFirst().get();
    System.out.printf("Première ligne du fichier : %s%n", firstLine);
} catch (NoSuchFileException e) {
    System.err.printf("Oups, fichier non trouvé : %s%n", e.toString());
} catch (AccessDeniedException e) {
    System.err.printf("Oups, pas la permission pour lire le fichier : %s%n", e.toString());
} catch (IOException e) {
    System.err.printf("Oups, une autre erreur d'I/O : %s%n", e.toString());
}
```

Attention ! Il faut songer à intercepter les exceptions dans le bon ordre : de la plus spécifique à la plus générique. Dans le code ci-dessus, essayez d'intercepter `IOException` en premier, et constatez l'erreur annoncée par votre IDE.

3.2 La mère de toutes les exceptions

Toutes les exceptions descendent de la classe `Exception`. Nous pourrions donc écrire

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            lancerMonProgramme();  
        } catch (Exception e) {  
            // ne faisons rien.  
        }  
    }  
}
```

et ne jamais voir la moindre exception. Ceci est pourtant une très mauvaise idée puisque cela cacherait tous les problèmes sans les gérer. Pour l'utilisateur final, cela implique que le programme s'arrête (sans message d'erreur) au moindre problème.

De façon générale nous déconseillons d'intercepter `Exception` silencieusement : si une erreur apparaît mais n'était pas prévue, votre programme risque de se retrouver dans un état incohérent. Il vaut mieux planter dès qu'un problème inconnu apparaît, afin de pouvoir le corriger, plutôt que de les cacher et ne plus comprendre leur origine : c'est le principe du *fail fast*¹

4 Mise en œuvre

Dans les deux TDs MVC, vous avez écrit des programmes, mais n'avez peut être pas rendu votre programme robuste :

- ▷ le modèle lève des exceptions, les avez-vous gérées ?
- ▷ la vue interagit avec l'utilisateur, mais que se passe-t-il si celui-ci ou celle-ci n'écrit pas exactement ce qui est attendu ?

Prenez le temps de rendre robuste votre travail.

1. Voir https://en.wikipedia.org/wiki/Fail-fast_system.